Pokémon: Shiny Vice

System Design Specification Document · Version 3.0 (Comprehensive Build)

1. Executive Summary & Core Design Pillars

Pokémon: Shiny Vice is an advanced, system-driven Pokémon Emerald decompilation modification designed from the ground up to elevate shiny hunting from a secondary, passive activity into a highly competitive, progression-gated core career path.

1.1 Economic Synergy: The macro loop binds player effort directly to visual prestige. Players must construct optimized utility teams, face high-difficulty tactical battles to secure currency, and spend resources to optimize real-world hunting math.

1.2 Transparency Mechanics: All probability statistics are calculated in real-time and fully exposed to the user layout via the interactive tracking interface. This removes abstract guessing parameters from the ecosystem.

2. Workspace Configuration & Compiler Dependencies

The project targets native compliance with modern Linux distributions (e.g., Nobara, Ubuntu environments running Windows Subsystem for Linux v2). Development utilizes standard GNU toolchain architectures compiled against the *RHH pokeemerald-expansion* source workspace.

Standard Toolchain Installation Terminal Commands:

```
$ sudo apt update
$ sudo apt install build-essential binutils-arm-none-eabi gcc-arm-none-eabi \
libnewlib-arm-none-eabi git libpng-dev devkitpro-arm-none-eabi
$ git clone https://github.com/rh-hideout/pokeemerald-expansion.git ShinyVice
$ cd ShinyVice && make -j$(nproc)
```

3. Additive Roll-Budget Engine & Permanent Struct Allocation

Unlike traditional binary modification platforms that apply raw percentage multipliers to standard base ratios, Shiny Vice utilizes an **Additive Roll-Budget Framework** clamped explicitly at a strict integer ceiling.

3.1 Mathematical Contract: The base shiny calculation generates an initial standalone odds ratio matching the Generation 6+ standard framework (1/4096). Dynamic multipliers (e.g., Tiered Shiny Charms, active chaining sequences) add linear calculation iterations inside a *for-loop* construct up to a maximum capped budget boundary of 8 total rolls, yielding a hard ceiling probability of 1/512.

src/pokemon.c — Restructured Core Rolling Logic:

```
bool8 IsPokemonShiny(struct Pokemon *mon)
{
u32 otId = GetMonData(mon, MON_DATA_OT_ID, NULL);
u32 personality = GetMonData(mon, MON_DATA_PERSONALITY, NULL);
u16 trainerId = otId & 0xFFFF;
```

```
u16 secretId = otId >> 16;
u8 totalRolls = 1;

if (CheckBagHasItem(ITEM_SHINY_CHARM_TIER_1, 1)) totalRolls += 1;
if (GetActiveChainBonus() > 0) totalRolls += GetActiveChainBonus();

if (totalRolls > 8) totalRolls = 8; // Enforce global design cap (1/512)

for (u8 i = 0; i < totalRolls; i++)
{
u32 shinyValue = (trainerId ^ secretId) ^ (personality ^ (personality >> 16));
if ((shinyValue & 0xFFF0) == 0) return TRUE;
personality = Random32(); // Generate pseudo-random entropy shift
}
return FALSE;
}
```

3.2 Save Allocation Safeguards: GBA architecture limits data structures to rigid, static sector maps. To track global persistent variables (currency indices, pity metrics, active map chain keys) across resets, variables are hardcoded directly into the permanent secondary sector memory space.

```
include/global.h — Structural SaveBlock2 Injections:
struct SaveBlock2
{
/* ... Vanilla data structures ... */
u32 shinyShardsCount; // Shard wallet balance tracking
u16 currentChainSpecies; // Active DexNav targeting index
u8 currentChainCount; // Active chain metric cache
u8 breedingPityCounter; // Structural dry-spell variable
};
```

4. Regional Pokédex Allocation & Utility Catcher Meta Hierarchy

To completely eliminate file bloating and mitigate severe save-block allocation crashes, Version 1 imposes a curated cross-generational regional index limit capped strictly between **400 and 500 species**.

4.1 Composition Ratio: The directory allocates a dominant 60% baseline to native Hoenn species to guarantee flawless visual compatibility with complete, pre-existing community overworld assets. The remaining 40% balances choice generational evolutions and starter sets scattered as rare event triggers.

4.2 Catching Meta Specifications Table:

Species	Target Role Assignment	Core Move Pool Logic
Breloom	Primary Status Enforcer	Spore (100% Accuracy Sleep) + False Swipe (1HP Clamp Recoil Protection)
Smeargle	Universal Utility Hazard Handler	Custom Sketch Routine: Thief (Held Items) + Soak (Type Conversion for Ghosts)
Spinda / Malamar	Alpha Encounter Dampener	Contrary Ability + Skill Swap Scripting (Inverts Self-Destructive Boss Buffs)

5. Two-Phase Starter Shunting UI Architecture

To address the conceptual balancing trade-off between modern gameplay optimization and retro ritualistic nostalgia, starter tracking mechanics are split across two separate game file architectures.

5.1 Fresh Save Files (The Purist Configuration): Enforces the physical requirement of execution reboots. The engine retains an optimized visual interface inspired by Radical Red, allowing immediate palette evaluation inside Professor Birch's bag UI before selection. True hardware real-time clock (RTC) entropy arrays are mapped natively to override vanilla Emerald's broken RNG seed tracking, completely preventing identical string duplicates across frames.

5.2 New Game Plus (NG+ Sandbox Modality): Gated as an earned unlock upon champion completion. The interface activates an **Instant Menu Re-roll Sub-Routine**. Pressing the *SELECT* or *B* key executes a script-level jump command that instantly updates active sprite parameters natively at 60–100 checks per minute without restarting the system.

5.3 Starter Pity Engine: Activated exclusively during New Game Plus playthroughs. An internal persistent counter registers consecutive menu inputs; crossing defined thresholds dynamically increases localized roll counts to completely eliminate catastrophic bad-luck dry spells for core starter aces.

6. Expanded DexNav Interface & Operational Chaining Data Rules

The tracking application is refactored from standard binary behavior to operate as an all-in-one data HUD screen displaying absolute live tracking data variables.

6.1 Interface HUD Specifications: Top Right tracks Search Levels via permanent global increment registers. The Center Grid displays visual attribute previews (Egg Moves, Hidden Ability flags, and IV Potential Stars). The Bottom Right updates the active dynamic Roll-Budget mathematical fraction layout in real-time.

6.2 Strict Elimination Criteria: To maintain the intrinsic value of the final shiny asset, wild chaining parameters impose hard termination conditions. An active streak is permanently zeroed out if the user selects *Flee/Run* from a targeted combat instance, accidentally defeats/captures a non-targeted species, crosses map zoning boundaries, or triggers a hard reset. Traversal maps integrate safety padding: fleeing from a random,

non-DexNav encounter while moving between active grass rustles does not break the chain tracker.

6.3 Ultimate NG+ Chain Lock: Upon initializing New Game Plus, the maximum chain metric permanently locks and clamps at 99 upon completion. Every subsequent search automatically loops at maximum calculation value, yielding an infinite 1/512 environment as long as zoning rules are maintained.

7. Horde Transactional Blocks & Poké Ball Safety Locks

Inspired by PokeMMO layout matrices, Horde encounters trigger an asymmetrical 1v3 or 1v5 combat environment mapped natively across Hoenn's geographic pathing vectors.

7.1 Balance Isolation: Hordes completely ignore active single-wild chaining values, drawing calculations exclusively from base odds plus global active charms to prevent extreme statistical decay. This serves as a hard balancing throttle.

7.2 Poké Ball Target Locking: The script logic locks capture item execution arrays completely until only a single wild target remains active on the field. This engineering choice protects internal active RAM indicators from selection indexing crashes and retains high tension during combat phases.

8. The Shiny Shard Economy Architecture

The entire intermediate and late-game scaling curve functions via a tightly regulated resource loop designed to systematically reward competitive strategy.

8.1 Accumulation Vectors: Shards are acquired exclusively via high-difficulty loops. Defeating or capturing dynamic wild Alpha entities rewards 1–2 shards. Completing high-difficulty daily Gym Leader full-rematch compositions distributes 5 shards. Clearing complete Battle Frontier facility brackets pays a baseline of 3 shards. If a player triggers a critical wild shiny fail (fleeing or accidental KO), the battle cleanup script verifies the sprite's shiny boolean value and deposits 25 Consolation Shards to cushion the failure loop.

8.2 Sinks and Progression Paths: Shards serve as the singular currency needed to purchase high-tier hunting multipliers, tiered shiny charms, Sweet Scent Ocarinas, targeted route locks, out-of-region starter eggs, and manual control registers on the central Weather/Season Terminal.